

MPO Convention Issue: FSM + build_mpo Operator Ordering

Summary

The existing FSM (Core/fsm.jl) and MPO builder (Builders/mpobuilder.jl) use a **lower-triangular** W-matrix convention. This silently **reverses** the operator ordering in two-site couplings: `FiniteRangeCoupling(:A, :B, 1, J)` produces $J * B_i * A_{i+1}$ instead of the expected $J * A_i * B_{i+1}$.

The bug is invisible for symmetric models (Heisenberg, XXZ) because swapping both operators in every term gives the same Hamiltonian. It will produce **wrong results** for any asymmetric coupling (e.g. Dzyaloshinskii–Moriya).

1. How the FSM Builds Transitions

`FiniteRangeCoupling(op1, op2, dx, weight)` is meant to represent:

$$H \supset \text{weight} \times \text{op1}_i \text{op2}_{i+dx}$$

In `_build_path` (Core/fsm.jl, lines 84–96):

```
function _build_path(ns::Int, coupling::FiniteRangeCoupling, transitions)
    path = ns+1 : ns+coupling.dx
    # op1 goes into transition (path[1], 1, ...)
    push!(transitions, (path[1], 1, coupling.op1, 1.0))
    # identity hops for dx > 1
    for i in 2:length(path)
        push!(transitions, (path[i], path[i-1], :I, 1.0))
    end
    # op2 goes into transition (0, path[end], ...)
    push!(transitions, (0, path[end], coupling.op2, coupling.weight))
    return path[end], transitions
end
```

After `build_FSM` remaps $0 \rightarrow \text{chi}$, the transitions become:

Transition	W-matrix entry	Location in W
$(\text{path}[1], 1, \text{op1})$	$W[\text{path}, 1] = \text{op1}$	First column
$(\text{chi}, \text{path}[\text{end}], \text{op2})$	$W[\text{chi}, \text{path}] = \text{op2}$	Last row

So `op1` lands in the **first column** and `op2` lands in the **last row**.

2. How build_mpo Extracts Boundaries

In `Builders/mpobuilder.jl` (lines 26–27):

```
L = reshape(bulk[chi, :, :, :], (1, chi, d, d))    # picks LAST ROW
R = reshape(bulk[:, 1, :, :], (chi, 1, d, d))    # picks FIRST COLUMN
```

This is the **lower-triangular** boundary convention:

- **Left boundary** selects the last row (row `chi`)
- **Right boundary** selects the first column (column 1)

3. Where the Reversal Happens

The MPO contracts left-to-right as a matrix product:

$$H = v_L^\top \cdot W_1 \cdot W_2 \cdots W_N \cdot v_R$$

With lower-triangular boundaries, `v_L` picks the last row and `v_R` picks the first column. For a nearest-neighbor coupling, the non-trivial path through the bond indices is:

Site	Bond path	W entry used
Site i	<code>chi → path</code>	<code>W[chi, path] = weight*op2</code>
Site i+1	<code>path → 1</code>	<code>W[path, 1] = op1</code>

The resulting coupling term is:

$$\text{weight} \times \text{op2}_i \times \text{op1}_{i+1}$$

The operators are swapped. The left-site operator comes from `op2` (the last row), and the right-site operator comes from `op1` (the first column).

4. Concrete Example

For $H = J \sum_i Y_i Z_{i+1}$:

```
coupling = FiniteRangeCoupling(:Y, :Z, 1, J)
```

The user expects: $J * Y_i * Z_{i+1}$

What the MPO actually encodes: $J * Z_i * Y_{i+1}$

Numerical verification (N=4, J=1.3, h=0.7):

Target: $H = J * Y_i Z_{i+1} + h X_i$

Swapped: $H = J * Z_i Y_{i+1} + h X_i$

Are they the same? false

`FiniteRangeCoupling(:Y, :Z, 1, J)` produces: $J * Z_i Y_{i+1}$ (WRONG)

This is masked in symmetric models because S_x*S_x , S_y*S_y , S_z*S_z are invariant under operator swap.

5. The Same Issue Affects Other Coupling Types

ExpChannelCoupling (lines 99–110): - $op1 \rightarrow W[path, 1]$ (first column, right site) - $op2 \rightarrow W[chi, path]$ (last row, left site) - Same reversal.

PowerLawCoupling (lines 112–124): - $op1 \rightarrow W[path[i], 1]$ (first column, right site) - $op2 \rightarrow W[chi, path[i]]$ (last row, left site) - Same reversal.

Field is unaffected (single-site operator, no ordering issue).

6. Suggested Fix: Switch to Upper-Triangular Convention

Change **only the boundary extraction** in `build_mpo` (`Builders/mpobuilder.jl`). No changes to the FSM are needed — the transitions naturally encode a left-to-right automaton.

Current code (lower-triangular):

```
# Left boundary: last row
L = reshape(bulk[chi, :, :, :], (1, chi, d, d))
# Right boundary: first column
R = reshape(bulk[:, 1, :, :], (chi, 1, d, d))
```

Fixed code (upper-triangular):

```
# Left boundary: first row
L = reshape(bulk[1, :, :, :], (1, chi, d, d))
# Right boundary: last column
R = reshape(bulk[:, chi, :, :], (chi, 1, d, d))
```

Why this works

With upper-triangular boundaries, `v_L` picks the **first row** and `v_R` picks the **last column**. The coupling path becomes:

Site	Bond path	W entry used
Site i	$1 \rightarrow path$	$W[1, path]$ — but this is zero in current layout

Wait — the FSM transitions populate $W[path, 1]$ (column 1) and $W[chi, path]$ (row `chi`). With upper-triangular boundaries picking row 1 and column `chi`, we need the transitions to populate **row 1** and **column `chi`** instead.

So the FSM transitions also need remapping. The cleanest single fix:

Option A: Swap op1/op2 in FSM _build_path methods

Change the three `_build_path` methods so that `op1` goes to the last-row transition and `op2` goes to the first-column transition. This preserves the lower-triangular boundary convention and makes `FiniteRangeCoupling(:A, :B, dx, w)` correctly produce $w * A_i * B_{\{i+dx\}}$.

In `_build_path` for `FiniteRangeCoupling`:

```
function _build_path(ns::Int, coupling::FiniteRangeCoupling, transitions)
    path = ns+1 : ns+coupling.dx
    # op2 goes to first column (right-site operator in lower-tri)
    push!(transitions, (path[1], 1, coupling.op2, 1.0))
    for i in 2:length(path)
        push!(transitions, (path[i], path[i-1], :I, 1.0))
    end
    # op1 goes to last row (left-site operator in lower-tri), with weight
    push!(transitions, (0, path[end], coupling.op1, coupling.weight))
    return path[end], transitions
end
```

Apply the same swap to `ExpChannelCoupling` and `PowerLawCoupling`.

Option B: Switch both FSM and boundary to upper-triangular

Remap the FSM so that: - The “start idle” state uses $W[1, 1] = I$ and left boundary picks row 1
- Coupling operators populate row 1 (left-site op) and column `chi` (right-site op) - The “end idle” state uses $W[\text{chi}, \text{chi}] = I$ and right boundary picks column `chi`

This requires changes to both `build_FSM` (state numbering and transition direction) and `build_mpo` (boundary extraction). More invasive but results in a fully consistent upper-triangular codebase matching `xxz_builder.jl`.

7. Recommendation

Option A (swap `op1/op2` in FSM) is the minimal fix — three small edits in `fsm.jl`, zero changes to `mpobuilder.jl`. It preserves backward compatibility for all symmetric Hamiltonians and fixes asymmetric ones.

Option B is cleaner architecturally — the entire codebase uses one convention — but requires more changes and testing.

Either way, add a regression test:

```
# Test: FiniteRangeCoupling(:Y, :Z, 1, J) must give J * Y_i * Z_{i+1}
# Compare FSM+build_mpo against Kronecker-product Hamiltonian for N=4
```

Files Affected

File	What to change
src/Core/fsm.jl	Swap op1 <-> op2 in <code>_build_path</code> (Option A), or remap state numbering (Option B)
src/Builders/mpobuilder.jl	No change (Option A), or swap boundary lines (Option B)
src/Builders/xxz_builder.jl	Already uses upper-triangular — no change needed